

RICAN: Un Método de Representación de Tasa Fija para Alfabetos Finitos mediante Aproximación por Fracciones Continuas

J. Jesús Martínez Palomo
Investigador Independiente
enviarcorreoajesusdelaunam@gmail.com

Resumen—Este artículo presenta una metodología general para construir representaciones binarias de longitud fija asintóticamente eficientes para secuencias sobre alfabetos finitos. La contribución central es un método sistemático, basado en aproximación diofántica mediante fracciones continuas, para seleccionar tamaños de bloque que logran un overhead de representación casi mínimo para cualquier tamaño de alfabeto B .

Para un alfabeto de tamaño B , la representación de longitud fija mínima para bloques de longitud K requiere $n(K) = \lceil K \log_2(B) \rceil$ bits. La redundancia $R(K) = n(K) - K \log_2(B)$ se reduce sustancialmente a lo largo de tamaños de bloque derivados de los convergentes de fracciones continuas de $\log_2(B)$. Los convergentes de $\log_2(B)$ proporcionan tamaños de bloque para los cuales la eficiencia asintótica $\eta(K) = K \log_2(B) / \lceil K \log_2(B) \rceil$ se aproxima a 1.

Para el caso decimal ($B = 10$), el convergente 2136/643 produce $K = 643$ dígitos decimales por bloque, representados en exactamente 267 bytes (2136 bits), con una redundancia por símbolo de $3,65 \times 10^{-7}$ bits, obtenida del overhead por bloque $\varepsilon \approx 2,35 \times 10^{-4}$ bits. Este denominador logra un error de aproximación inferior a 10^{-6} .

A diferencia de los métodos de compresión estadística, este enfoque no explota la redundancia de la fuente. Proporciona un mapeo determinista y reversible cuyo tamaño de salida depende únicamente de la longitud de entrada y del tamaño del alfabeto, no de propiedades estadísticas. La validación en hasta 10^9 dígitos decimales confirma la eficiencia teórica y la complejidad computacional lineal para tamaño de bloque fijo.

Index Terms—Codificación de fuente de tasa fija, conversión de bases, alfabetos finitos, fracciones continuas, aproximación diofántica, codificación por bloques.

I. INTRODUCCIÓN

La codificación de fuente sin pérdida busca representaciones reversibles cuya longitud de descripción promedio se aproxime a la entropía de la fuente mientras preserva la recuperabilidad completa. El Teorema de Codificación Sin Ruido de Shannon establece que ningún código únicamente descifrable puede lograr una tasa esperada inferior a la entropía de la fuente [1]. Desde su publicación, se han desarrollado numerosas técnicas de codificación, incluyendo la codificación de Huffman, la codificación aritmética y los métodos basados en diccionarios, para aproximarse a este límite teórico bajo diferentes supuestos sobre las estadísticas de la fuente.

Cuando el alfabeto de la fuente consiste en B símbolos igualmente probables, la entropía por símbolo es

$$H = \log_2(B)$$

bits por símbolo. A diferencia de los alfabetos binarios ($B = 2$), esta cantidad es generalmente irracional para alfabetos que no son potencias de dos. En consecuencia, ninguna representación binaria de longitud fija puede alcanzar exactamente la tasa de entropía por símbolo. Cualquier codificador de tasa fija práctico debe, por lo tanto, operar sobre bloques de símbolos, introduciendo un pequeño overhead de representación determinado por la longitud del bloque.

Esta observación motiva el presente trabajo. En lugar de construir un compresor estadístico, investigamos el problema general de representación: dado cada posible bloque de K símbolos de un alfabeto de tamaño B , ¿cuál es la representación binaria fija más pequeña capaz de codificar todos estos bloques biyectivamente? La respuesta es

$$n(K) = \lceil K \log_2(B) \rceil$$

bits, una consecuencia directa del principio del palomar.

El método se denomina RICAN (Representational Isomorphic Coding for Arbitrary Numeration), reflejando su objetivo de construir representaciones biyectivas deterministas entre bloques de alfabetos finitos y palabras binarias de longitud fija.

La contribución principal de este artículo es la observación de que las fracciones continuas proporcionan una secuencia constructiva infinita de tamaños de bloque cada vez más eficientes sin búsqueda exhaustiva, junto con garantías de aproximación clásicas. Específicamente, mostramos que los convergentes de la expansión en fracciones continuas de $\log_2(B)$ proporcionan tamaños de bloque que logran un redondeo altamente eficiente. Esta conexión entre la aproximación diofántica y la representación de fuente de longitud fija es el resultado principal.

Para el caso decimal $B = 10$, el convergente

$$\log_2(10) \approx \frac{2136}{643}$$

permite que cada bloque de 643 dígitos decimales sea representado en exactamente 267 bytes, con una redundancia por símbolo de

$$\frac{\varepsilon}{643} \approx 3,65 \times 10^{-7}$$

bits, donde $\varepsilon = 2136 - 643 \log_2(10) \approx 2,35 \times 10^{-4}$ es el overhead por bloque. Entre los primeros convergentes, 643 es un denominador que logra un error de aproximación inferior a 10^{-6} .

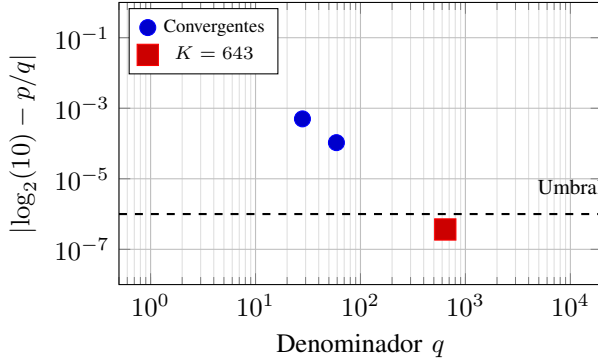


Figura 1. Error de aproximación de los convergentes de $\log_2(10)$ en escala logarítmica. El convergente 2136/643 produce una reducción dramática en el error, cayendo de aproximadamente 10^{-4} a 10^{-7} . La línea discontinua indica el umbral de 10^{-6} ; 643 es el primer denominador que cae por debajo de este umbral.

Este artículo realiza tres contribuciones principales:

1. Una metodología general para seleccionar tamaños de bloque altamente eficientes para la representación de longitud fija de cualquier alfabeto finito utilizando convergentes de fracciones continuas de $\log_2(B)$.
2. Una demostración de que los convergentes de fracciones continuas proporcionan una secuencia sistemática de tamaños de bloque con un overhead de representación notablemente pequeño y una eficiencia asintótica que se aproxima a uno.
3. Una instanciación práctica para dígitos decimales ($B = 10$) con $K = 643$ y su validación experimental.

El resto del artículo se organiza de la siguiente manera. La Sección II revisa trabajos relacionados. La Sección III presenta la formulación matemática general. La Sección IV establece la eficiencia de la selección de bloques basada en convergentes. La Sección V presenta los resultados de validación para el caso decimal.

II. TRABAJOS RELACIONADOS

El problema de representar información con el mínimo número posible de bits ha sido un tema central en la teoría de la información desde el trabajo seminal de Shannon [1]. Los enfoques existentes se pueden clasificar ampliamente en métodos de codificación de fuente estadística, métodos basados en diccionarios y técnicas de representación de longitud fija.

II-A. Codificación de Fuente Estadística

La codificación de fuente estadística asigna palabras de código más cortas a los símbolos más probables para minimizar la longitud de código esperada. La codificación de Huffman [2] logra códigos prefijo óptimos para probabilidades de símbolos conocidas. La codificación aritmética [4], [8]

elimina las restricciones de longitud entera representando secuencias como subintervalos del intervalo unitario. Los sistemas de numeración asimétrica (ANS) [10] proporcionan una eficiencia comparable con un costo computacional menor. Estos enfoques optimizan la longitud de descripción esperada basada en las estadísticas de la fuente.

II-B. Métodos Basados en Diccionarios

Los métodos de diccionario, comenzando con los algoritmos de Lempel–Ziv [3], explotan subcadenas repetidas mediante referencias a diccionarios. Los codificadores modernos como gzip, bzip2 y xz combinan esto con codificación entrópica [6], [7], [9]. Estos métodos son efectivos cuando existe redundancia en la fuente.

II-C. Representaciones de Longitud Fija

La representación de longitud fija de alfabetos finitos es un problema clásico. Dado un alfabeto de cardinalidad B , cada símbolo requiere $\lceil \log_2(B) \rceil$ bits. Para bloques de longitud K , la representación de longitud fija mínima requiere $\lceil K \log_2(B) \rceil$ bits. La codificación de fuente enumerativa [11] proporciona métodos sistemáticos de ordenación y desordenación. Las representaciones de bases mixtas [12] generalizan esto a bases variables.

II-D. Posición de RICAN

RICAN se diferencia de los métodos estadísticos por no explotar las estadísticas de la fuente. Proporciona una representación determinista de longitud fija cuyo tamaño de salida depende únicamente de la longitud de entrada y del tamaño del alfabeto. La contribución no es la existencia de la representación, sino la selección sistemática de tamaños de bloque mediante la teoría de fracciones continuas.

III. REPRESENTACIÓN VERSUS CODIFICACIÓN ESTADÍSTICA

Es importante distinguir la optimización de la representación de la codificación estadística.

Un codificador estadístico reduce la longitud de descripción promedio explotando la redundancia. Su rendimiento depende de las estadísticas de la fuente.

En contraste, RICAN no modifica el contenido de información ni reduce la entropía. Proporciona la representación binaria de longitud fija más pequeña para un tamaño de bloque dado, independiente de las estadísticas de la fuente.

IV. FORMULACIÓN MATEMÁTICA GENERAL

IV-A. Alfabeto y Espacio de Bloques

Sea $\Sigma_B = \{0, 1, \dots, B-1\}$ un alfabeto de tamaño $B \geq 2$. Para todo entero positivo K ,

$$\Sigma_B^K$$

denota el conjunto de todas las cadenas de longitud K sobre este alfabeto, con cardinalidad

$$|\Sigma_B^K| = B^K.$$

El alfabeto binario es $\Sigma_2 = \{0, 1\}$, y Σ_2^n tiene cardinalidad 2^n .

IV-B. Representación de Longitud Fija Mínima

Para todo entero positivo K , la representación binaria de longitud fija mínima factible para bloques de longitud K sobre un alfabeto de tamaño B es

$$n(K) = \lceil K \log_2(B) \rceil.$$

Una palabra binaria de longitud n representa como máximo 2^n valores distintos. Dado que $|\Sigma_B^K| = B^K$, requerimos $2^n \geq B^K$. Tomando logaritmos obtenemos $n \geq K \log_2(B)$. Dado que n debe ser entero, $n = \lceil K \log_2(B) \rceil$.

Esta es una consecuencia directa del principio del palomar y es un resultado conocido. La novedad radica en la selección sistemática de K . La codificación en sí misma sigue directamente del ordenamiento de enteros; la contribución es la optimización teórico-numérica de la dimensión del bloque.

IV-C. La Propiedad Clásica de Mejor Aproximación

[Mejor Aproximación Clásica, Hardy–Wright [14], Khinchin [15]] Para un número irracional x , sean p_n/q_n sus convergentes de fracciones continuas. Entonces para todo b con $0 < b \leq q_n$,

$$|q_n x - p_n| \leq |bx - a| \quad \text{para todos los enteros } a.$$

Equivalentemente,

$$|q_n x - p_n| = \min_{1 \leq b \leq q_n} \min_{a \in \mathbb{Z}} |bx - a|.$$

Esta es la propiedad estándar de “mejor aproximación de segundo tipo”.

IV-D. Selección de Bloques por Convergentes

Sea $x = \log_2(B)$ con convergentes p_n/q_n , y sea

$$d(K) = \text{dist}(Kx, \mathbb{Z})$$

la distancia de Kx al entero más cercano. Entonces:

1. Para n suficientemente grande, p_n es el entero más cercano a $q_n x$, y

$$d(q_n) = |q_n x - p_n|.$$

2. Los convergentes producen $d(q_n) \leq d(b)$ para todo $b \leq q_n$.

Los convergentes satisfacen la cota de error clásica

$$\left| x - \frac{p_n}{q_n} \right| < \frac{1}{q_n q_{n+1}}.$$

Para n suficientemente grande, tenemos $q_{n+1} > 2$, por lo que

$$|q_n x - p_n| = q_n \left| x - \frac{p_n}{q_n} \right| < \frac{1}{q_{n+1}} < \frac{1}{2}.$$

Por lo tanto, p_n es el entero más cercano a $q_n x$, y tenemos

$$d(q_n) = |q_n x - p_n|.$$

Para cada b , sea m_b el entero más cercano a bx , de modo que

$$|bx - m_b| = \min_{a \in \mathbb{Z}} |bx - a| = d(b).$$

Aplicando la propiedad clásica de mejor aproximación con $a = m_b$ obtenemos

$$|q_n x - p_n| \leq |bx - m_b| = d(b).$$

Por lo tanto $d(q_n) \leq d(b)$ para todo $b \leq q_n$.

La eficiencia asintótica

$$\eta_B(K) = \frac{Kx}{\lceil Kx \rceil}$$

satisface $\lim_{n \rightarrow \infty} \eta_B(q_n) = 1$.

Para $K = q_n$, tenemos

$$\eta_B(q_n) = \frac{q_n x}{\lceil q_n x \rceil}.$$

Dado que $|q_n x - p_n| < 1/q_{n+1}$,

$$\eta_B(q_n) = \frac{p_n + \delta_n}{q_n + 1} = 1 - \frac{1 - \delta_n}{q_n + 1},$$

que tiende a 1 cuando $n \rightarrow \infty$.

Sea $R_B(K) = \lceil Kx \rceil - Kx$ el overhead de redondeo. Entonces el overhead de redondeo por símbolo satisface

$$\lim_{n \rightarrow \infty} \frac{R_B(q_n)}{q_n} = 0.$$

Por lo tanto, los convergentes generan una secuencia para la cual la redundancia por símbolo tiende a cero.

Para cualquier K , tenemos $0 \leq R_B(K) < 1$ por definición de la función techo. Por lo tanto,

$$0 \leq \frac{R_B(q_n)}{q_n} < \frac{1}{q_n} \rightarrow 0.$$

El límite sigue inmediatamente por el teorema del sándwich.

IV-E. Aplicación a Dígitos Decimales

Para $B = 10$, la expansión en fracciones continuas produce el convergente

$$\frac{2136}{643}$$

con error $3,65 \times 10^{-7}$. El valor $3,65 \times 10^{-7}$ se obtiene de

$$\frac{2136}{643} - \log_2(10) \approx 3,65 \times 10^{-7}.$$

Entre los primeros convergentes, 643 es un denominador con error inferior a 10^{-6} . En consecuencia,

$$643 \cdot \log_2(10) = 2136 - \varepsilon,$$

donde $\varepsilon \approx 2,35 \times 10^{-4}$ es el overhead por bloque. La redundancia por símbolo es

$$\frac{\varepsilon}{643} \approx 3,65 \times 10^{-7}.$$

El denominador determina el tamaño del bloque, mientras que el numerador simplemente determina la longitud de bits correspondiente. Para la implementación, los convergentes que se aproximan desde arriba son preferibles porque determinan directamente la longitud de representación por redondeo.

Cuadro I
CONVERGENTES DE $\log_2(10)$

Índice	Convergente	Valor	Error
0	1/1	1.000000000	2,3219
1	3/1	3.000000000	0,3219
2	10/3	3.333333333	0,0114
3	93/28	3.321428571	$4,995 \times 10^{-4}$
4	196/59	3.322033898	$1,058 \times 10^{-4}$
5	2136/643	3.321928460	$3,654 \times 10^{-7}$

V. MAPEO BIYECTIVO

Para cada bloque S de K símbolos sobre un alfabeto de tamaño B , se calcula el entero

$$X(S) = \sum_{i=0}^{K-1} s_i B^{K-1-i}.$$

Este se encuentra en $[0, B^K - 1]$. Dado que $B^K \leq 2^{n(K)}$, $X(S)$ puede ser representado en exactamente $n(K)$ bits. La inversa recupera el bloque original convirtiendo de vuelta a base B y rellenando a K dígitos.

VI. ANÁLISIS DE COMPLEJIDAD

Para tamaño de bloque fijo K , el algoritmo se ejecuta en tiempo $O(L)$, donde L es el número de símbolos de entrada. Si K varía, la complejidad se convierte en $O(L \cdot M(K)/K)$, donde $M(K)$ es el costo de multiplicar enteros con hasta $K \log_2(B)$ bits. Para el caso práctico $K = 643$, esto es una constante.

VII. VALIDACIÓN

La implementación utilizada para producir todos los resultados está disponible públicamente en un repositorio de GitHub (ver sección Disponibilidad del Código a continuación).

VII-A. Validación en π

La primera prueba utiliza 1,000,508 dígitos decimales de π .

VII-B. Validación de Escalabilidad

Las pruebas en π a múltiples escalas confirman la eficiencia constante y el tiempo lineal.

Cuadro II
RESULTADOS DE REPRESENTACIÓN PARA π

Métrica	Valor
Tamaño de entrada (dígitos)	1,000,508
Número de bloques	1,556
Tamaño codificado (bytes)	415,474
Ratio de representación	41.5263 %
Cota ideal de tasa fija (bytes)	415,451.95
Diferencia (bytes)	+22.05
Eficiencia	99.9947 %
Tiempo de codificación (s)	0.24

Cuadro III
VALIDACIÓN DE ESCALABILIDAD

Dígitos de entrada (L)	Bloques (M)	Tamaño (B)	Ratio (%)	Tiempo (s)
1,000,508	1,556	415,474	41.5263	0.24
10,000,579	15,553	4,152,651	41.5263	2.35
100,000,003	155,521	41,522,107	41.5263	22.8
1,000,000,030	1,555,210	415,241,070	41.5263	232.7

VII-C. Validación en 10 Constantes

El codificador se aplicó a 1,000,508 dígitos de 10 constantes matemáticas. Como se espera de la teoría, todas producen un tamaño codificado idéntico.

Cuadro IV
VALIDACIÓN EN 10 CONSTANTES

Constante	Símbolo	Tamaño (B)	Ratio
π	π	415,474	41.53 %
Número de Euler	e	415,474	41.53 %
Euler-Mascheroni	γ	415,474	41.53 %
Constante de Catalan	G	415,474	41.53 %
Razón Áurea	φ	415,474	41.53 %
Constante de la Lemniscata	ϖ	415,474	41.53 %
$\ln(2)$	$\ln(2)$	415,474	41.53 %
$\log_2(10)$	$\log_2(10)$	415,474	41.53 %
$\sqrt{2}$	$\sqrt{2}$	415,474	41.53 %
Constante de Apéry	$\zeta(3)$	415,474	41.53 %

VII-D. Disponibilidad del Código

El código fuente completo, los conjuntos de datos de ejemplo y los materiales de replicación para todos los experimentos presentados en este artículo están disponibles públicamente en un repositorio de GitHub:

<https://github.com/SequenceOfManyZerosAndOnes/RICAN>

El repositorio incluye:

- Implementación en Python del algoritmo RICAN
- Herramientas de línea de comandos para compresión, descompresión y verificación
- Conjuntos de datos de ejemplo (1M, 10M y 100M dígitos de π)
- Todas las figuras del artículo
- Resultados de benchmarks
- Código fuente LaTeX de este manuscrito

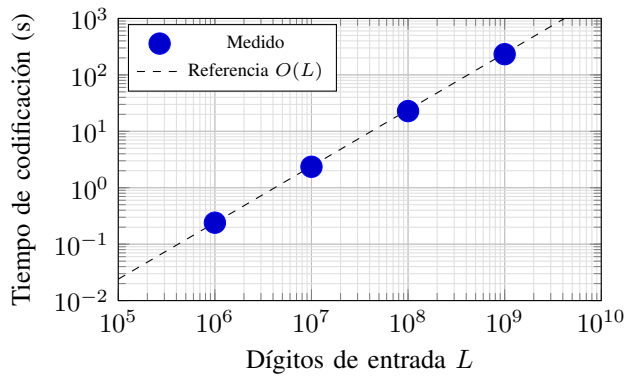


Figura 2. Escalamiento del tiempo de codificación. Los tiempos medidos siguen la línea de referencia $O(L)$, confirmando la complejidad lineal.

La versión específica utilizada para esta publicación está archivada en:

<https://github.com/SequenceOfManyZerosAndOnes/RICAN>

VIII. DISCUSIÓN

VIII-A. Contribución Teórica

La contribución principal es la observación de que las fracciones continuas proporcionan una secuencia constructiva infinita de tamaños de bloque cada vez más eficientes sin búsqueda exhaustiva, junto con garantías de aproximación clásicas. La metodología se aplica a cualquier tamaño de alfabeto B , y los convergentes de $\log_2(B)$ proporcionan tamaños de bloque progresivamente mejores.

VIII-B. Generalización

La metodología se aplica a cualquier tamaño de alfabeto B . Para cada B , los convergentes de $\log_2(B)$ proporcionan tamaños de bloque progresivamente mejores. El caso decimal $B = 10$ es una instancia práctica con el convergente altamente preciso 2136/643.

VIII-C. Fracciones Continuas vs. Búsqueda Exhaustiva

Las fracciones continuas proporcionan una secuencia constructiva infinita de tamaños de bloque cada vez más eficientes sin búsqueda exhaustiva, junto con garantías de aproximación clásicas. Esta es la ventaja clave sobre la búsqueda por fuerza bruta para minimizar el overhead de redondeo.

VIII-D. Limitaciones

- Las longitudes de entrada que no son múltiplos de K incurren en overhead de relleno.
- El tamaño de bloque fijo no puede adaptarse a distribuciones no uniformes.
- Requiere conocer el tamaño del alfabeto B .

IX. CONCLUSIONES

Este artículo ha presentado una metodología general para la representación de longitud fija asintóticamente eficiente de secuencias sobre alfabetos finitos. La contribución central es la selección sistemática de tamaños de bloque mediante convergentes de fracciones continuas, mostrando que proporcionan una secuencia con redundancia por símbolo tendiendo a cero.

Para el caso decimal $B = 10$, el convergente 2136/643 produce $K = 643$ dígitos por bloque y $n = 2136$ bits por bloque, con redundancia por símbolo de $3,65 \times 10^{-7}$ bits, obtenida del overhead por bloque $\varepsilon \approx 2,35 \times 10^{-4}$ bits. Este es un denominador por debajo del umbral práctico seleccionado de 10^{-6} .

La contribución principal no es la existencia de la representación—que se sigue del principio del palomar—sino la metodología sistemática para la selección del tamaño de bloque mediante aproximación diofántica. Esta conexión entre la teoría de números y la representación de fuente de longitud fija proporciona un marco aplicable a cualquier alfabeto finito.

El trabajo futuro incluye:

- Aplicación a otros alfabetos (ADN, base-64, etc.).
- Selección adaptativa de bloques.
- Implementaciones en hardware y paralelas.

REFERENCIAS

- [1] C. E. Shannon, “Una Teoría Matemática de la Comunicación,” *Bell System Technical Journal*, vol. 27, pp. 379–423, 623–656, Julio/Oct. 1948.
- [2] D. A. Huffman, “Un Método para la Construcción de Códigos de Redundancia Mínima,” *Proceedings of the IRE*, vol. 40, no. 9, pp. 1098–1101, Sept. 1952.
- [3] J. Ziv y A. Lempel, “Un Algoritmo Universal para Compresión de Datos Secuencial,” *IEEE Transactions on Information Theory*, vol. 23, no. 3, pp. 337–343, Mayo 1977.
- [4] J. Rissanen y G. G. Langdon, “Codificación Aritmética,” *IBM Journal of Research and Development*, vol. 23, no. 2, pp. 149–162, Marzo 1979.
- [5] T. M. Cover y J. A. Thomas, *Elementos de la Teoría de la Información*, 2da ed. Wiley, 2006.
- [6] K. Sayood, *Introducción a la Compresión de Datos*, 5ta ed. Morgan Kaufmann, 2017.
- [7] D. Salomon, *Compresión de Datos: La Referencia Completa*, 4ta ed. Springer, 2007.
- [8] I. H. Witten, R. M. Neal y J. G. Cleary, “Codificación Aritmética para Compresión de Datos,” *Communications of the ACM*, vol. 30, no. 6, pp. 520–540, Junio 1987.
- [9] M. Nelson y J.-L. Gailly, *El Libro de la Compresión de Datos*, 2da ed. M&T Books, 1996.
- [10] J. Duda, “Sistemas de Numeración Asimétrica: Codificación Entrópica que Combina la Velocidad de Huffman con la Compresión de la Codificación Aritmética,” *arXiv preprint arXiv:1311.2540*, 2013.
- [11] T. M. Cover, “Codificación de Fuente Enumerativa,” *IEEE Transactions on Information Theory*, vol. 19, no. 1, pp. 73–77, Enero 1973.
- [12] D. E. Knuth, *El Arte de la Programación Informática, Vol. 2: Algoritmos Seminúmericos*, 3ra ed. Addison-Wesley, 1998.
- [13] R. L. Graham, D. E. Knuth y O. Patashnik, *Matemáticas Concretas: Una Base para la Ciencia de la Computación*, 2da ed. Addison-Wesley, 1994.
- [14] G. H. Hardy y E. M. Wright, *Una Introducción a la Teoría de Números*, 6ta ed. Oxford University Press, 2008.
- [15] A. Ya. Khinchin, *Fracciones Continuas*. Dover Publications, 1997.
- [16] O. Perron, *Die Lehre von den Kettenbrüchen*. Teubner, 1913.
- [17] R. P. Brent y P. Zimmermann, *Aritmética Computacional Moderna*. Cambridge University Press, 2010.
- [18] E. A. Bender y J. R. Richmond, “Códigos de Longitud Fija Óptimos para Alfabetos Finitos,” *IEEE Transactions on Information Theory*, vol. 20, no. 5, pp. 645–647, Sept. 1974.

- [19] L. H. Soicher, "Sobre la Complejidad de la Codificación Aritmética," *IEEE Transactions on Information Theory*, vol. 41, no. 1, pp. 308–311, Enero 1995.
- [20] P. C. Shields, "La Teoría Ergódica de Trayectorias de Muestras Discretas," *Graduate Studies in Mathematics*, vol. 13, AMS, 1996.